

Bloque B2

Servicio B — Recurso (CRUD de tareas)

Laboratorio. Construirás el segundo servicio con el CRUD completo de tareas. Por ahora SIN protección: la añadiremos en B3. Duración aproximada: 2.5 horas.

Qué vas a construir

El **Servicio B** corre en el puerto **3002** (distinto al de A) y administra el recurso tareas con las cuatro operaciones CRUD: listar, crear, actualizar y eliminar. Es un servicio independiente: su propio proyecto, su propio `package.json`, su propio `.env`.

Comparte la base de datos con el Servicio A (atajo de laboratorio). En un sistema real, cada servicio tendría su propia base; aquí compartimos `awos_u3` para simplificar el montaje.

Paso 0 — Crear el proyecto

Desde la carpeta `servicio-recurso`. Nota que aquí **no** instalamos `bcrypt`: este servicio no maneja contraseñas.

```
cd servicio-recurso
npm init -y
npm install express mysql2 jsonwebtoken cors dotenv
```

Paso 1 — Construye el servicio fragmento por fragmento

Fragmento 1 — Variables de entorno (.env)

Archivo: `servicio-recurso/.env`

Copia `.env.example` a `.env`. El puerto es **3002**. El `JWT_SECRET` debe ser **exactamente el mismo** que pusiste en el Servicio A; lo necesitaremos en B3 para verificar tokens.

```
PORT=3002
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=
DB_NAME=awos_u3
JWT_SECRET=cadena-larga-aleatoria-igual-en-ambos-servicios
```

Fragmento 2 — Conexión a MySQL (igual que en A)

Archivo: `servicio-recurso/src/db.js`

El pool de conexiones es idéntico al del Servicio A. Apunta a la misma base, pero este servicio solo trabajará con la tabla `tareas`.

```
const mysql = require('mysql2/promise');
require('dotenv').config();

const pool = mysql.createPool({
  host: process.env.DB_HOST,
  port: process.env.DB_PORT,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
});

module.exports = pool;
```

Fragmento 3 — Listar y crear tareas (GET y POST)

Archivo: servicio-recurso/src/tareas.routes.js

GET /tareas devuelve la lista con 200. **POST /tareas** inserta y responde **201** con la tarea creada. Fíjate que el resultado de execute se desestructura como `const [resultado] = ...: mysql2` siempre devuelve un arreglo cuyo primer elemento son las filas (en SELECT) o la metadata del INSERT (con `insertId`).

Por ahora usamos un `usuario_id` fijo de prueba (1). En B3 lo tomaremos del token; recuérdalo.

```
const express = require('express');
const pool = require('./db');
const router = express.Router();

router.get('/tareas', async (req, res, next) => {
  try {
    const [filas] = await pool.execute(
      'SELECT id, titulo, completada, creado_en FROM tareas ORDER BY id DESC');
    res.json(filas);
  } catch (err) { next(err); }
});

router.post('/tareas', async (req, res, next) => {
  try {
    const { titulo } = req.body;
    const [resultado] = await pool.execute(
      'INSERT INTO tareas (titulo, usuario_id) VALUES (?, ?)', [titulo, 1]);
    res.status(201).json({ id: resultado.insertId, titulo, completada: false });
  } catch (err) { next(err); }
});
```

Fragmento 4 — Actualizar y eliminar (PUT y DELETE)

Archivo: servicio-recurso/src/tareas.routes.js

PUT /tareas/:id usa `req.params.id` (viene en la ruta) para saber qué tarea actualizar. Revisamos `affectedRows`: si es 0, no existía esa tarea, así que respondemos **404**.

DELETE /tareas/:id responde **204** (sin contenido) cuando borra, o 404 si no había nada que borrar. 204 es el código correcto para 'éxito, no tengo nada que devolverte'.

```

router.put('/tareas/:id', async (req, res, next) => {
  try {
    const { titulo, completada } = req.body;
    const [r] = await pool.execute(
      'UPDATE tareas SET titulo = ?, completada = ? WHERE id = ?',
      [titulo, completada ? 1 : 0, req.params.id]);
    if (r.affectedRows === 0) return res.status(404).json({ error: 'Tarea no encontrada' });
    res.json({ id: Number(req.params.id), titulo, completada: !!completada });
  } catch (err) { next(err); }
});

router.delete('/tareas/:id', async (req, res, next) => {
  try {
    const [r] = await pool.execute('DELETE FROM tareas WHERE id = ?', [req.params.id]);
    if (r.affectedRows === 0) return res.status(404).json({ error: 'Tarea no encontrada' });
    res.status(204).send();
  } catch (err) { next(err); }
});

module.exports = router;

```

Fragmento 5 — Armar el servidor

Archivo: servicio-recurso/src/index.js

Mismo patrón que el Servicio A: cors, json, rutas y manejador de errores al final. Agregamos una ruta pública /salud para comprobar rápido que el servicio vive. En B3 insertaremos el middleware de autenticación entre el json y las rutas de tareas.

```

const express = require('express');
const cors = require('cors');
require('dotenv').config();
const tareasRoutes = require('./tareas.routes');

const app = express();
app.use(cors());
app.use(express.json());

app.get('/salud', (req, res) => res.json({ servicio: 'recurso', estado: 'ok' }));
app.use('/', tareasRoutes);

app.use((err, req, res, next) => {
  console.error('[RECURSO][ERROR]', err);
  res.status(500).json({ error: 'Error interno del servidor' });
});

const PORT = process.env.PORT || 3002;
app.listen(PORT, () => console.log(`Servicio B en http://localhost:${PORT}`));

```

Paso 2 — Probar

Arranca el servicio desde su carpeta y prueba el CRUD completo con tu cliente REST. Como aún no hay protección, cualquiera puede llamar a estos endpoints; eso cambia en B3.

```
cd servicio-recurso
node src/index.js
# GET    http://localhost:3002/tareas
# POST   http://localhost:3002/tareas    body: { "titulo": "Probar" }
# PUT    http://localhost:3002/tareas/1   body: { "titulo": "Editar", "completada": true }
# DELETE http://localhost:3002/tareas/1
```

Recordatorio de seguridad

Igual que en el Servicio A, todas las consultas usan `execute` con `?` (prepared statements). Aunque el dato venga de la ruta (`:id`), nunca lo metas directo en el texto del SQL: un id manipulado podría intentar inyección. Los placeholders te protegen.

Resultado esperado de este bloque

Un CRUD de tareas funcional con los códigos correctos: 200 al listar, 201 al crear, 200/404 al actualizar, 204/404 al eliminar. Sin autenticación todavía — ese es el trabajo de B3, donde estos dos servicios por fin se conectan.